



C++ PROGRAMMING

(335)

NATIONAL - 2019

Production Portion:

Job 1: Golden Oaks _____ (380 points)

TOTAL POINTS _____ ***(380 points)***

Graders: Please double check and verify all scores and answer keys!

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.

C++ PROGRAMMING - NATIONAL 2019
ANSWER KEY
Page 2 of 10



```
// Golden Oaks.cpp : Defines the entry point for the console application.

// By: Bryan J. Baker - BakerLand Labs

//#include "stdafx.h"

#include <iostream>

#include <iomanip>

#include <cstdlib>

#include <ctime> // needed for rand()% seed

#include <string>

#include <vector>

#include <fstream>

#include <conio.h>

using namespace std;

const int NUM_COURSES = 6;

// Structer to hold the rounds

struct Round

{

    vector <int> NineHole_Rounds;

    vector <int> EighteenHole_Rounds;

};

// Structer to hold Member info and Rounds

struct Member

{

    string Name;

    int ClubID;

    vector <Round> Course_Holes;

    vector <double> Hndc;

};

//Struct to hold a Course's Data
```

struct Course

```
{
    string Name;

    double Rating;

    int Slope;
};

// Var Section
ifstream In_File;

string Test;

int X, Y, Size;

bool Flag;

// Vectors to hold Courses and Members
//- Must use Literals for size values

vector <Course> Courses(6);

vector <Member> Members(4);

// Prototypes for Methods Section

void Read_File(vector<Course> &, vector <Member> &);

int Calc_Course_Handicap(vector <int>, vector <int>, int, int);

int sort(vector <Member> &, int);

void menu();

void main()
{
    system("CLS");

    Read_File(Courses, Members);

    // Calc and store Handicaps for each member and courses

    for (int M = 0; M < Members.size(); M++)
    {
        Members[M].Hndc.resize(Courses.size());

        for (X = 0; X < Courses.size(); X++)
```



C++ PROGRAMMING - NATIONAL 2019

ANSWER KEY

Page 4 of 10



```
                Members[M].Hndc[X] =
Calc_Course_Handicap(Members[M].Course_Holes[X].NineHole_Rounds,
                    Members[M].Course_Holes[X].EighteenHole_Rounds, Courses[X].Rating,
Courses[X].Slope);

    }

    menu();

}

// Read the data in from the files

void Read_File(vector<Course> & Courses, vector <Member> & Members)

// Note you MUST have a space after &

{

    int I, Num_Rounds, M, Course, R;

    string Test;

    In_File.open("Golf Data.txt");

    if (!In_File.fail())

    {

        // Read the Course data in based on the constant used for the number of courses

        for (I = 0; I < NUM_COURSES; I++)

        {

            getline(In_File, Courses[I].Name);

            In_File >> Courses[I].Rating;

            In_File >> Courses[I].Slope;

            getline(In_File, Test); // use to move to the next line

        } // End Collecting Courses

        M = 0;

        do {

            Members.resize(M + 1);

            getline(In_File, Members[M].Name);

            In_File >> Members[M].ClubID;

            Members[M].Course_Holes.resize(NUM_COURSES);

            // Read the rounds for each Member of all 6 courses
```



```
        for (Course = 0; Course < NUM_COURSES; Course++)
        {
            // Reading The Courses 9 hole rounds for that Member
            In_File >> Num_Rounds;
Members[M].Course_Holes[Course].NineHole_Rounds.resize(Num_Rounds);

            for (R = 0; R < Num_Rounds; R++)
                In_File >> Members[M].Course_Holes[Course].NineHole_Rounds[R];

            // Reading The Courses 18 hole rounds for that Member
            In_File >> Num_Rounds;
Members[M].Course_Holes[Course].EighteenHole_Rounds.resize(Num_Rounds);

            for (R = 0; R < Num_Rounds; R++)
                In_File >> Members[M].Course_Holes[Course].EighteenHole_Rounds[R];
        }

        getline(In_File, Test); // use to move to the next line

        M++;

    } while (!In_File.eof()); // End collecting Members
}

else // File is missing so stop the program
{
    cout << "File is missing. Press enter to close program." << endl;
    getchar();
    exit(1);
}

In_File.close();
}

// Use the formula to calculate the Course Handicap

int Calc_Course_Handicap(vector <int> Nine_Holes, vector <int> Eighteen_Holes,
    int Course_Rating, int Slope_Rating)
{
    int Handicap = 0;
```



```
double Mgs = 0;

int I, D;

// Figure out which way to calculate the MGS

// Use 18 holes

if (Eighteen_Holes.size() >= 5)
{
    for (I = 0; I < Eighteen_Holes.size(); I++)
        Mgs += Eighteen_Holes[I];

    Mgs /= Eighteen_Holes.size();
}

// Use 9 Holes

else if (Nine_Holes.size() >= 10)
{
    for (I = 0; I < Nine_Holes.size(); I++)
        Mgs += (Nine_Holes[I] * 2);

    Mgs /= Nine_Holes.size();
}

else // Need both to get the MGS
{
    if ((Nine_Holes.size() + Eighteen_Holes.size()) > 20)
        D = 20;

    else
        D = Nine_Holes.size() + Eighteen_Holes.size();

    for (I = 0; I < Nine_Holes.size(); I++)
        Mgs += (Nine_Holes[I] * 2);

    for (I = 0; I < Eighteen_Holes.size(); I++)
        Mgs += Eighteen_Holes[I];

    Mgs /= D;
}
```



```

    }

    Mgs = ((Mgs - Course_Rating) * 113) / Slope_Rating;

    // Round the value

    Handicap = Mgs;

    if ((Mgs - Handicap) > 0.5)

        Handicap++;

    return Handicap;

}

// Pass in the Members list by reference and which Course to display by value
// This sort is the Bubble Sort

int sort(vector <Member> & Members, int Course)

{

    int I, J;

    Member Temp;

    int Longest = 0;

    for (I = 0; I < Members.size(); I++)

    {

        for (J = 0; J < Members.size() - I - 1; J++)

            if (Members[J].Hndc[Course] < Members[J + 1].Hndc[Course])

            {

                Temp = Members[J];

                Members[J] = Members[J + 1];

                Members[J + 1] = Temp;

            }

    }

    // Find the longest Name and use that for displaying data

    for (I = 0; I < Members.size(); I++)

        if (Members[I].Name.length() > Longest)

            Longest = Members[I].Name.length();

```



```
        return Longest;
    }

    // Method to display the menu
    void menu()
    {
        int Value = 0;

        int I, Setw_Size, Spacing;

        string Trash;

        char Ans;

        do {
            do {
                system("CLS");

                cout << setw(60) << "Golden Oaks Membership Handicap List" << endl << endl;

                for (I = 0; I < Courses.size(); I++)
                    cout << setw(10) << I + 1 << ":" << Courses[I].Name << endl;

                cout << setw(10) << I + 1 << ":Member List" << endl;

                cout << setw(10) << I + 2 << ":Exit" << endl;

                cout << endl << setw(50) <<
                    "Please pick the course number to display or exit and press enter.";

                cin >> Value;

                getline(cin, Trash);
                // Error check user input
                if (Value < 0 || Value > Courses.size())
                {
                    cout << endl << setw(50) <<
                        "Please enter only the number 0 -" << Courses.size() << "." << endl;

                    cout << setw(45) << "Press ENTER to continue." << endl;

                    // hold message on the screen until the user presses Enter
                    getchar();
                }
            } while (Value < 1 || Value > (Courses.size() + 2));
        }
```



```
// To clear the buffer so the getchar works below.
getline(cin, Trash);
if (Value != Courses.size() + 2)
{
    Value--;
    // Display Courses
    if (Value < Courses.size())
    {
        system("CLS");
        cout << endl << "Course:" << Courses[Value].Name << endl << endl;
        Setw_Size = sort(Members, Value);
        for (I = 0; I < Members.size(); I++)
        {
            if (I < 9)
                cout << " " << I + 1;
            else
                cout << I + 1;
            cout << "." << setiosflags(ios::right)
                << setw(Members[I].Name.length()) << Members[I].Name;
            if (Members[I].Name.length() > 14)
                Spacing = 15;
            else
                Spacing = 30 - Members[I].Name.length();
            cout << setw(Spacing) << "Handicap:" <<
                Members[I].Hndc[Value] << endl;
        }
    }
    // Display Members
```



```
        else
        {
            system("CLS");
            cout << setw(60) << "Golden Oaks Membership List" << endl;
            cout << endl;
            for (I = 0; I < Members.size(); I++)
            {
                if (I < 9)
                    cout << " " << I + 1;

                else
                    cout << I + 1;

                cout << ". " << setiosflags(ios::right) <<
                    setw(Members[I].Name.length()) << Members[I].Name;
                if (Members[I].Name.length() > 14)
                    Spacing = 15;

                else
                    Spacing = 30 - Members[I].Name.length();

                cout << setw(Spacing) << "Member #:" << Members[I].ClubID
                    << endl;
            }
        }

        cout << endl << "Please press ENTER to return to options menu."
            << endl;

        getchar();
    }

    // Repeat while not Exit
} while (Value != (Courses.size() + 2));
}
```